Fitting Data to Models

Sean Bryan
Arizona State University

1 The Average as a "Best Fit Model"

Before getting into the weeds with math, it is worthwhile to step back and motivate a fairly common sense result. Why is taking the average of a group of numbers a good idea?

Concretely, let's say we measured the height in inches of each of a group of people, and put them into an array called x:

$$x = [67, 61, 64, 62, 60]. \tag{1}$$

Here the entire collection of heights is x, and the i-th item in the array is x_i. So in this case (starting counting from zero like the python programming language does) x_2 is 64.

The simplest possible model for this data would be to assert that there really is a single "true" height h and that the variations from person to person we see are just random. We need to determine what number we should put in for h in our model.

Whatever h we choose should have the smallest possible difference from the data. To make the math work out better later in the process, let us calculate the total difference between the model and our data by calculating the sum of the squares of the differences. (In math, physics and statistics this quantity mysteriously gets called χ^2, that is "chi squared," and in machine learning and AI it gets called the "cost function," so to make the later equations cleaner let's call this quantity C.) This is:

$$C(h) = \sum_{i=0}^{N-1} (x_i - h)^2 \tag{2}$$

where N is the number of elements in the array x. Writing this out verbosely for the case of our array of five heights, this is:

$$C(h) = (67 - h)^2 + (61 - h)^2 + (64 - h)^2 + (62 - h)^2 + (60 - h)^2. \tag{3}$$

A plot of this function for our particular case is shown in Figure 1

The best value for h would be the one that has the smallest error $C(h)$. By eye in Figure 1 this looks like it is a little below 63. Note that the derivative of this function C with respect to the model parameter h is zero at this minimum. So, if we want a formula for this minimum, we should calculate dC/dh, set it equal to zero, and solve for h.

Taking the derivative of the verbose version, and setting it equal to zero, yields:

$$\frac{dC}{dh} = -2(67 - h) - 2(61 - h) - 2(64 - h) - 2(62 - h) - 2(60 - h) = 0. \tag{4}$$

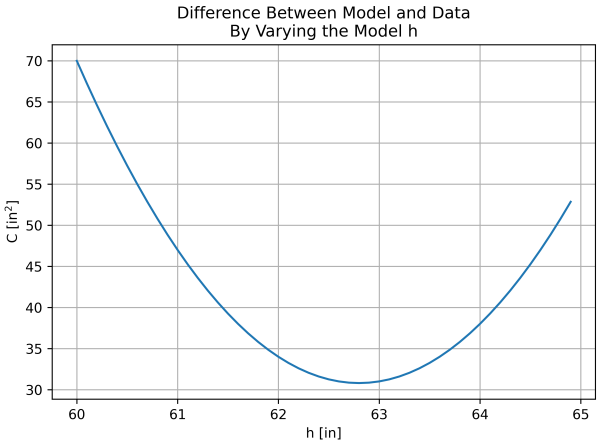

Figure 1: Plot of the sum of the squares of (model-data) for the case of the height data.

Diividing both sides by -2 for clarity, collecting all of the h values on one side, and the number values on the other, yields:

$$(67 + 61 + 64 + 62 + 60) = 5h \tag{5}$$

That leaves $h = (67 + 61 + 64 + 62 + 60)/5 = 62.8$ inches.

In the abstract, let's return to the original problem where we had in general N numbers in the array x. Taking the derivative of the fancy formula for C back in Equation 2 yields:

$$\frac{dC}{dh} = -2\sum_{i=0}^{N-1}(x_i - h) = 0. \tag{6}$$

Dividing by -2 again, and moving the x_i terms to one side, and the h terms to the other, yields

$$\sum_{i=0}^{N-1} x_i = \sum_{i=0}^{N-1} h. \tag{7}$$

Adding up the constant values on the right yields $N \times h$, so solving for h yields

$$h = \frac{\sum_{i=0}^{N-1} x_i}{N}. \tag{8}$$

This formula is just the formula for taking the mean of an array, that is it asks us to add up all the values in x and divide by N. But this derivation shows that the mean is the value that has the smallest difference to all of the individual elements in a data set. It is the "best-fit" model of a dataset that you model as being truly a single number, plus some random fluctuations.

2 Linear Fitting

The math is a little messier, but this same approach can be used to adjust model parameters to minimize the difference C between a model and data even for models that have many

parameters. (Indeed, this is how generative AI models are trained, their billions of parameters are adjusted to minimize the difference between the text the model puts out and the training data the creator of the model has access to.)

Following the treatment in Garcia's "Numerical Methods for Physics" and in Bevington and Robinson's "Data Reduction and Error Analysis for the Physical Sciences," let's derive and use the formula for a linear fit. The data now has both x and y arrays, each with N datapoints, as plotted in Figure 2. As a concrete example, consider that each x value is the number of items on a plate at a restaurant, and each y value is the price of that plate (in dollars).

$$\begin{array}{l} x = [\quad 1, \quad 2, \quad 4, \quad 5, \quad 7\ ] \\ y = [\ 3.49, \quad 4.62, \quad 8.53, \quad 10.51, \quad 15.32\ ] \end{array} \tag{9}$$

We now assert that our model y^{model} for the price is that there is a constant fixed price for serving the plate to the table, plus a per-item cost for each item on the plate. That, is our model is $y^{model} = mx + b$ where b is the fixed price for serving the plate, and m is the per-item cost. We have to select values for m and b in our model.

Just as before, we write out the sum of the squared of the difference between our model and the data. For this case, that is

$$C(m,b) = \sum_{i=0}^{N-1} (y_i - y_i^{model})^2 = \sum_{i=0}^{N-1} \left(y_i - (mx_i + b)\right)^2 \tag{10}$$

It is harder to visualize, but we will vary both m and b to find the combination of both that yields the lowest $C(m,b)$, and this combination of m and b will be the best-fit. At this two-dimensional minimum, the derivative of $C(m,b)$ with respect to m, and also with respect to b, both will be zero. Calculating that out yields

$$\begin{aligned} \frac{dC}{dm} &= -2\sum_{i=0}^{N-1} \left(y_i - (mx_i + b)\right) x_i = 0 \\ \frac{dC}{db} &= -2\sum_{i=0}^{N-1} \left(y_i - (mx_i + b)\right) = 0. \end{aligned} \tag{11}$$

This looks messy, but as before let's divide everything by -2 for clarity, and collect some terms.

$$\begin{aligned} \sum_{i=0}^{N-1} y_i x_i - m\sum_{i=0}^{N-1} x_i x_i - b\sum_{i=0}^{N-1} x_i &= 0 \\ \sum_{i=0}^{N-1} y_i - m\sum_{i=0}^{N-1} x_i - \sum_{i=0}^{N-1} b &= 0. \end{aligned} \tag{12}$$

It still looks messy, but we can compute some of the sums over the data values x and y "offline" and give them names. Let's define several variables, and add the actual numerical value for each for this particular dataset. Define $Sx \equiv \sum x_i = 19$, $Sy \equiv \sum y_i = 42.47$, $Sxy \equiv \sum x_i y_i = 206.64$, and $Sxx \equiv \sum x_i x_i = 95$. Again, these are each just a single

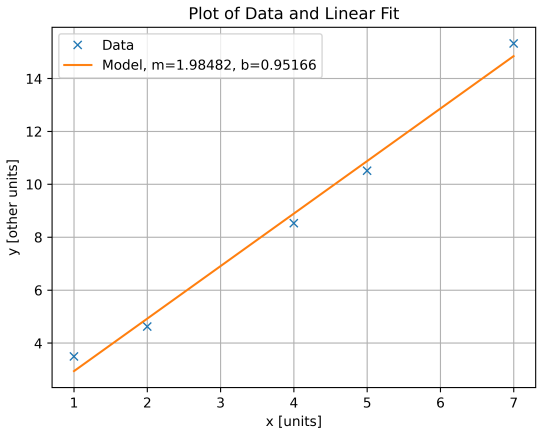

Figure 2: Plot of data and a linear fit.

number, not an array or function or anything else complicated like that. This lets us rewrite the equations as

$$\begin{aligned} Sxy &= m \times Sxx + b \times Sx \\ Sy &= m \times Sx + b \times N. \end{aligned} \tag{13}$$

This is a "two equations, two unknowns" situation, so leaving the details of the solution aside, yields

$$\begin{aligned} m &= \frac{N \times Sxy - Sy \times Sx}{N \times Sxx - (Sx)^2} \\ b &= \frac{Sy \times Sxx - Sx \times Sxy}{N \times Sxx - (Sx)^2}. \end{aligned} \tag{14}$$

Plugging in the specific numbers for our case yields the actual best-fit slope and offset values for our specific case

$$\begin{aligned} m &= \frac{5 \times 206.64 - 42.47 \times 19}{5 \times 95 - (19)^2} \approx 1.985 \\ b &= \frac{42.47 \times 95 - 19 \times 206.64}{5 \times 95 - (19)^2} \approx 0.952. \end{aligned} \tag{15}$$

Plotting this line over the data on Figure 2 shows that indeed the line does go through the data well.

Stepping back, this is a minor miracle! After some calculus, we were able to add together different combinations of the numbers in our data to get the best fit line. No AI needed! As you can read in the literature in the "Further Work" section, this approach can be further generalized to any function that is a linear combination of templates. That includes polynomials, waves of different frequencies, templates of measured data, or a wide range of other situations.

3 Computer Program

Again, this can all be calculated by hand or with a calculator, there is no magic here. You can also plot the data and the model by hand on graph paper. However, it is very convenient with larger datasets to have a computer program to calculate these formulas automatically, and plot the data automatically.

At the end of this document is a listing for a python program that does all of this. Read it, see how it implements the equations, type it in, and try running it to reproduce Figure 2. Try putting your own different data in for x and y and see what you get.

4 Further Work

Garcia's book "Numerical Methods for Physics" has an excellent section about fitting models to data. The documentation for `scipy` also has great information. In addition, `scikit-learn` has a powerful and usable collection of machine learning/neural network tools. On your own, look into general linear fitting (Garcia's book has a great derivation), non-linear fitting (the function `curve_fit` in `scipy` is a great tool for this and has good documentation, or `emcee` has a much fancier approach), and neural networks (start by going to `playground.tensorflow.org` and interacting with it, then later read up on `scikit-learn` neural network regression and classification and try examples).

Listing 1: Listing of a python program to do a linear fit. Note that the line numbers in grey at the left are for reference, and should not be typed in to the program.

```
1  import numpy as np
2  import matplotlib.pyplot as pl
3
4  # create the data arrays x and y
5  x = np.array([1,2,4,5,7])
6  y = np.array([3.49,4.62,8.53,10.51,15.32])
7
8  # calculate the sum quantities
9  N = len(x)
10 Sx = np.sum(x)
11 Sy = np.sum(y)
12 Sxx = np.sum(x*x)
13 Sxy = np.sum(x*y)
14
15 # calculate the best-fit slope and offsets
16 m = (N*Sxy  - Sy*Sx)  / (N*Sxx - Sx**2)
17 b = (Sy*Sxx - Sx*Sxy) / (N*Sxx - Sx**2)
18
19 # make a plot of the data and the fit
20 # create the figure
21 pl.figure()
22 # plot the actual two traces
23 pl.plot(x,y,'x',label='Data')
24 pl.plot(x,m*x + b,'-',label='Model, m='+str(m)+', b='+str(b))
25 # add labels to the axes
26 pl.xlabel('x [units]')
27 pl.ylabel('y [other units]')
28 # add a title
29 pl.title('Plot of Data and Linear Fit')
30 # show the legend
31 pl.legend()
32 # add a grid
33 pl.grid('on')
34 # show it to the screen
35 pl.ion()
36 pl.show()
```